Софийски университет „Св. Климент Охридски“
Факултет по математика и информатика

Курсов проект по Системи за е-бизнес
зимен семестър 2024/2025г.

Тема на проекта: Хотел

Изработил: Стелиана Мирославова Димитрова

Специалност: Информационни системи

Курс: 4

Факултетен номер: 4MI0700044

Гр. София
Януари 2025

Глава 1. Увод

1.Описание на идеята на проекта

2.Архитектура и дизайн на приложението

3.Описание на е-бизнеса

4.Функционалности

5.Потребителски случаи

6.UML диаграми на класовете

7.Описание на базата данни

8.Описание на потребителската кошница

9.Описание на плащането

10.Стартиране на проекта

Гр. София
Януари 2025

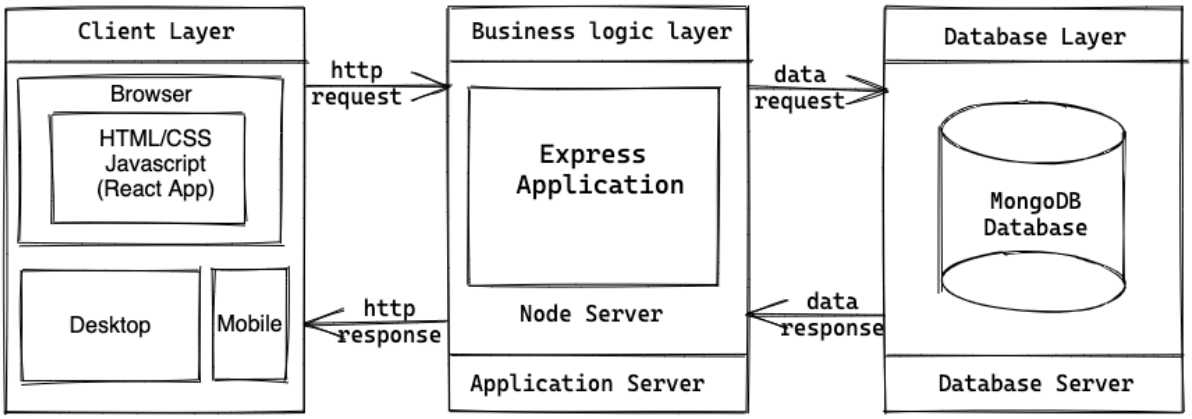
Глава 1. Увод

1. Описание на идеята на проекта

В днешно време наличието на професионален уебсайт за хотел е задължително, за да се гарантира неговият успех. Това доведе до нуждата да се създаде проектът Grand Wailea. Курсовият проект представя уеб платформа за управление на хотелски бизнес с цел да се предостави система за преглед на всички стаи и зали в един хотел, възможност за търсене по дати, управление на потребителски резервации, обработка на плащания и интеграция с външни системи – платежна система и система за получаване на точната локация на хотела. Той е разделен на 2 части – backend и frontend, за да се покаже границата между дизайна и интерактивността, които потребителите виждат, и логиката на приложението, заедно с базата данни и сървъра. По този начин се улеснява добавянето на нови функционалности, както и се подобрява бързото отстраняване на грешки, което в реална система е изключително важно. В проекта се гарантира висока производителност и интуитивен потребителски интерфейс за удобство на потребителите.

2. Архитектура и дизайн на приложението

За направата на системата съм използвала архитектурата MERN – MongoDB, Express.js, React, Node.js. Тя е базирана на разделяне на отговорностите между клиентската част, сървърната логика и базата данни. Това позволява лесно добавяне на нови функционалности.



Системата следва многослойна архитектура и по-точно 3-слойна архитектура (3-tier Architecture), която се състои от следните слоеве:

1.Представителен слой (Frontend)

- Той е създаден с React, който е отговорен за създаването на динамичен потребителски интерфейс. В основата му стои използването на компоненти, което позволява преизползването на код, и управлението на състоянието им.
- Създава се маршрутизиране с React Router за поддръжка на SPA (Single Page Application).
- Позволява използването на готови React компоненти.

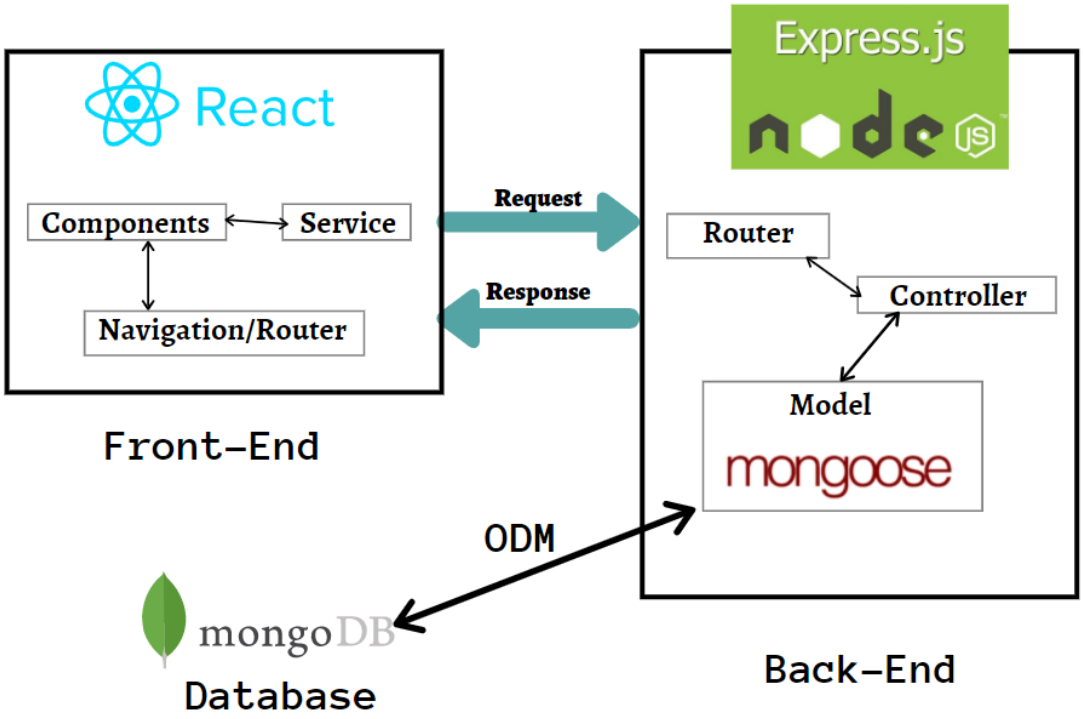
- Използван е Axios за изпращане на HTTP заявки към backend
- Използван е CSS за стилизиране.

2.Сървърен слой (Backend)

- Изграден е чрез Node.js и Express.js за предоставяне на RESTful API. Node.js слуша за заявки, изпраща отговори и осъществява връзката с базата, а Express.js се грижи за маршрутизацията и обработката на междинния слой, той улеснява server-side и логиката.
- Осигурява логиката за управление на потребители, обработка на поръчки и комуникация с базата данни.
- Използвани са JWB (JSON Web Tokens) за автентикация, криптиране на паролите чрез bcrypt и политики за CORS за ограничаване на достъпа до Stripe Api и разрешаване на взаимодействие между различните домейни.
- Разделен е на models-controllers-routes, за улеснение на управлението.

3. Слой за данни (Database)

- Използвана е MongoDB, която е NoSQL база данни за съхранение на данните в JSON формат. Предимството и е запазването на документи в колекции вместо доклади в таблици както е в SQL.
- Важен момент е свързването на базата със сървъра, а не с React, защото ако директно се свърже с фронтенда, идентификационните данните (credentials), ще бъдат изложени в client-side code, което ще доведе до огромен проблем със сигурността.
- Използван е Mongoose за дефиниране на модели, управление на схеми и заявки. Той позволява лесна манипулация на данните.



Гр. София
Януари 2025

3. Описание на е-бизнеса

В съвременният свят пътувания са по-достъпни от всякога, а конкуренцията в туристическия сектор е огромна. Всеки хотел се стреми да привлече повече клиенти, да ги убеди да изберат точно тяхното място, да изпъкнат с добър архитектурен дизайн и с лесна и интуитивна система, която е User Friendly. Именно за да се направи добро първо впечатление беше създаден сайт Grand Wailea, който да отговори на очакванията на потребителите.

Създадената платформа представлява уеб базирана система за резервации в хотел, която предоставя онлайн каталог за наличните помещения в сградата, избор на желаната стая и безпроблемна резервация. Предлага се галерия от снимки и описания, които да помогнат на потребителя да избере желаното от него място за престой, както и информация за връзка с рецепционистите, при всякакви въпроси. Системата предоставя локация и карта, която потребителите да използват, в случай че искат да видят точното местоположение на хотела (показано е с маркер) или да видят пешеходно къде се намира и как да се ориентират по улиците (добавено е човече за разходка).

Всеки един клиент ще може да направи свой собствен профил, в който да се запазват всички резервации, които е направил, и по този начин ще следи за предстоящите си почивки. Резервациите ще могат да се направят директно през платформата без да се начислява такса, както е по другите хотели.

Заинтересованите лица, които съм идентифицирала са хората от управлението на хотела (генерален мениджър, маркетингов отдел, отдел „Резервации“ и IT отделът, отговарящ за техническата поддръжка), персонала (собственици на стаи, спа център), туристическите агенции и клиентите. Крайният продукт, който ще отговори на очакванията им, е не само визуално привлекателен, а и функционален.

За бъдещи подобрения ще бъдат определени техните изисквания чрез въпросници, интервюта, както и прототипи на системата, за да се извлече максимално детайлна обратна връзка.

Системата е създадена да отговаря на нуждите на Grand Wailea хотел, който желае да привлича нови клиенти от всяка точка по света. Основната и цел е да улесни процесът за резервация за всички.

Анализ:

Направих анализ на вече съществуващия сайт на хотела и установих, че имат красив дизайн, но интерактивността му може понякога да натовари потребителя. Затова в новонаправената платформа съм заложена на семпъл и изчистен дизайн, който да удовлетвори всеки гост.

Също така нямат своя собствена резервация, а препращат към основният сайт на Hilton (тъй като Grand Wailea Maui е част от веригата на Hilton – A Waldorf Astoria Resort). В новонаправената платформа съм разрешила този проблем като потребителят директно може да направи резервацията и да заплати през системата, без допълнителни препращания и попълване отново на данни.

4. Функционалности

a. Функционални изисквания

- Опция за регистрация като клиент на системата след попълване на необходимите данни
- Опция за вход в системата след въвеждане на валидни данни (потребителско име и парола)
- В профила на потребителя да се показват всички направени резервации. Ако няма такива, тя трябва да изведе подходящи съобщения
- Потребителят може да търси и преглежда помещенията, предлагани от хотела
- Потребителят може да филтрира наличните стаи по дати и брой гости чрез сърч бар (поле, в което да се въведат check-in и check-out датите, на които потребителят иска да посети хотела)
- Потребителят преглежда отделна страница на всяка една стая.
- Потребителят въвежда датите на желаният престой във формата на конкретна зала
- Автоматично изчисление на общата цена за престоя във формата в страницата на конкретна стая
- Добавяне в количката
- Обновяване на наличността в количката
- При добавяне в количката, автоматично се променя броя (на артикулите в нея), който е изписан в навигационното поле
- В количката се запазват продуктите и след излизане от профила
- Може да се добави повече от 1 помещение наведнъж
- В страницата на количката автоматично се пресмята общата цена на всички резервации в кошницата като е написана и конкретната за всяка
- Потребител може да плати през системата след въвеждане на необходимите данни
- След успешно плащане се занулява количката
- Резервациите се запазват в профила на потребителят
- Достъп до социалните платформи на хотела – Facebook, X, Instagram

- Нерегистриран потребител има достъп до преглед на помещения, информация, снимки и локация.
- Нерегистриран потребител не може да прави резервации
- Нерегистриран потребител не може да добавя в количка
- Нерегистриран потребител не може да направи плащане
- Когато един потребител не е регистриран в лявата част на навигационното поле вижда само бутон за вход (Login) – не се визуализира профил и потребителска кошница.

b. Нефункционални изисквания

- Системата е съвместима с различни браузъри
- Системата използва bcrypt за хеширане на пароли
- Системата използва JWT за създаване на токъни, които да се използват за удостоверяване и ауторизация
- Времето за неактивност е 1 част
- Обработката на плащания се извършва чрез Stripe
- Системата поддържа обработка на множество маршрути (routes in backend), което оптимизира бързото време за отговор

Гр. София
Януари 2025

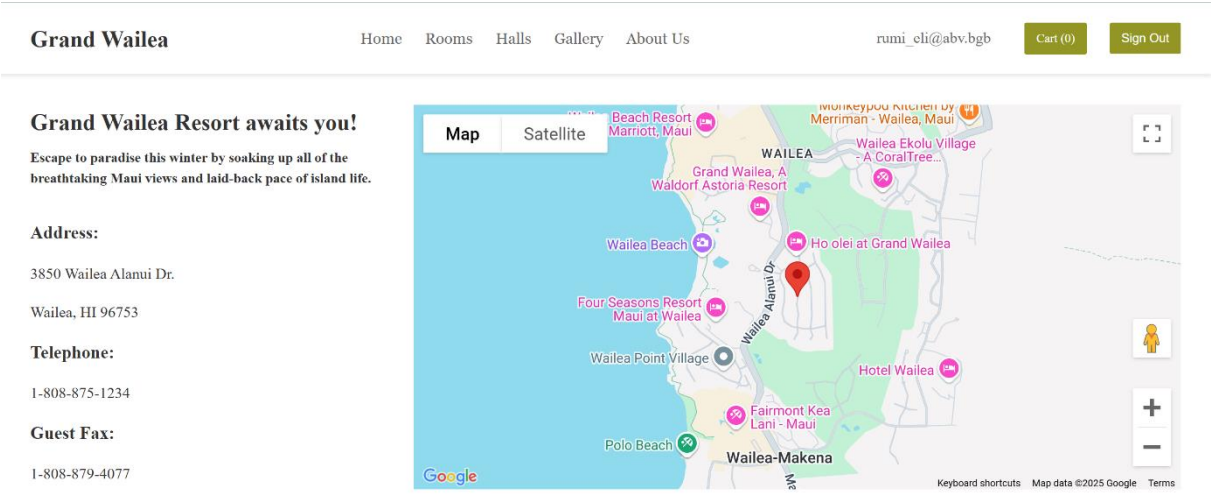
- Конфигурацията на средата е в .env, което предпазва чувствителните данни
- Всички маршрути и логики са защитени срещу неоторизиран достъп.
- React позволява бърза реализация на промените в потребителски интерфейс
- Поддържа се responsive design
- Използван е MongoDB, гарантиращ постоянство на данните
- Архитектурата е модулна (разделяне на маршрути, контролери и модели), което улеснява добавянето на нови функционалности.

c. Интеграция с външни системи

В проекта са интегрирани 2 външни системи. Това е важна част от съвременните уеб приложения, защото позволява да се използват услуги и функционалности, които не са вътрешно разработени.

1) Интеграция с Google Maps API

-Използва се за визуализация на адреса
- Включва интерактивна карта в страницата
    - Предлага преглед на картата като map или satellite
    - Точни координати на хотела
    - Увеличаване на цял екран
    - Промяна на мащаба – увеличаване/намаляване
    - Възможност за поставяне на човече за разходка из локацията
    - Пин/Маркер с точното местоположение
- Позволява лесна ориентация за местоположението на хотела
- Съвместимо е с различните устройства

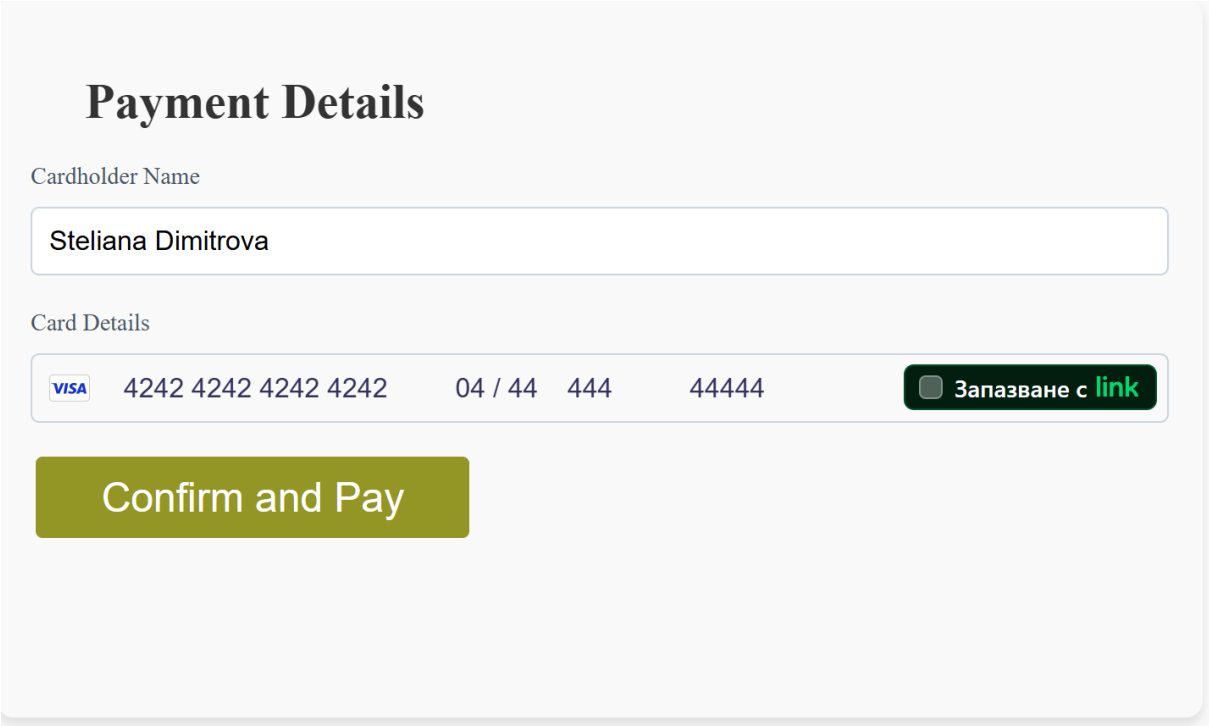


2) Интеграция със Stripe

- Използва се за обработка на онлайн плащания
- Предлага надеждна и сигурна обработка на транзакции

Гр. София
Януари 2025

- В проекта се използва за обработка на плащания за резервации на хотелски стаи
- Потребителят въвежда данните си за плащане, които се изпращат директно към Stripe чрез защитена връзка, а не се съхраняват в системата.
- Включва Stripe Elements за визуализация на платежния формуляр (Frontend)
- Използва Stripe SDK за създаване на плащания и валидиране на транзакции (Backend)
- API ключовете за Stripe се съхраняват в .env файл, за да се предпази чувствителната информация (тази информация не се обработва от сървъра)
- Stripe поддържа голям обем от транзакции



5. <u>Потребителски случаи</u>

За представянето на проекта съм изготвила модел на потребителските случаи, който визуализира и помага за разбирането на това как потребителите взаимодействат със системата и по какъв начин участват външните системи.

Чрез описание на потребителските случаи подробно съм дефинирала всяка стъпка от взаимодействието.

Представила съм ги под формата на диаграма и текстово описание.

За описание на потребителските случаи съм изготвила таблица с описание на потребителските случаи в кратък формат. Подредени са по приоритет – high, medium, low

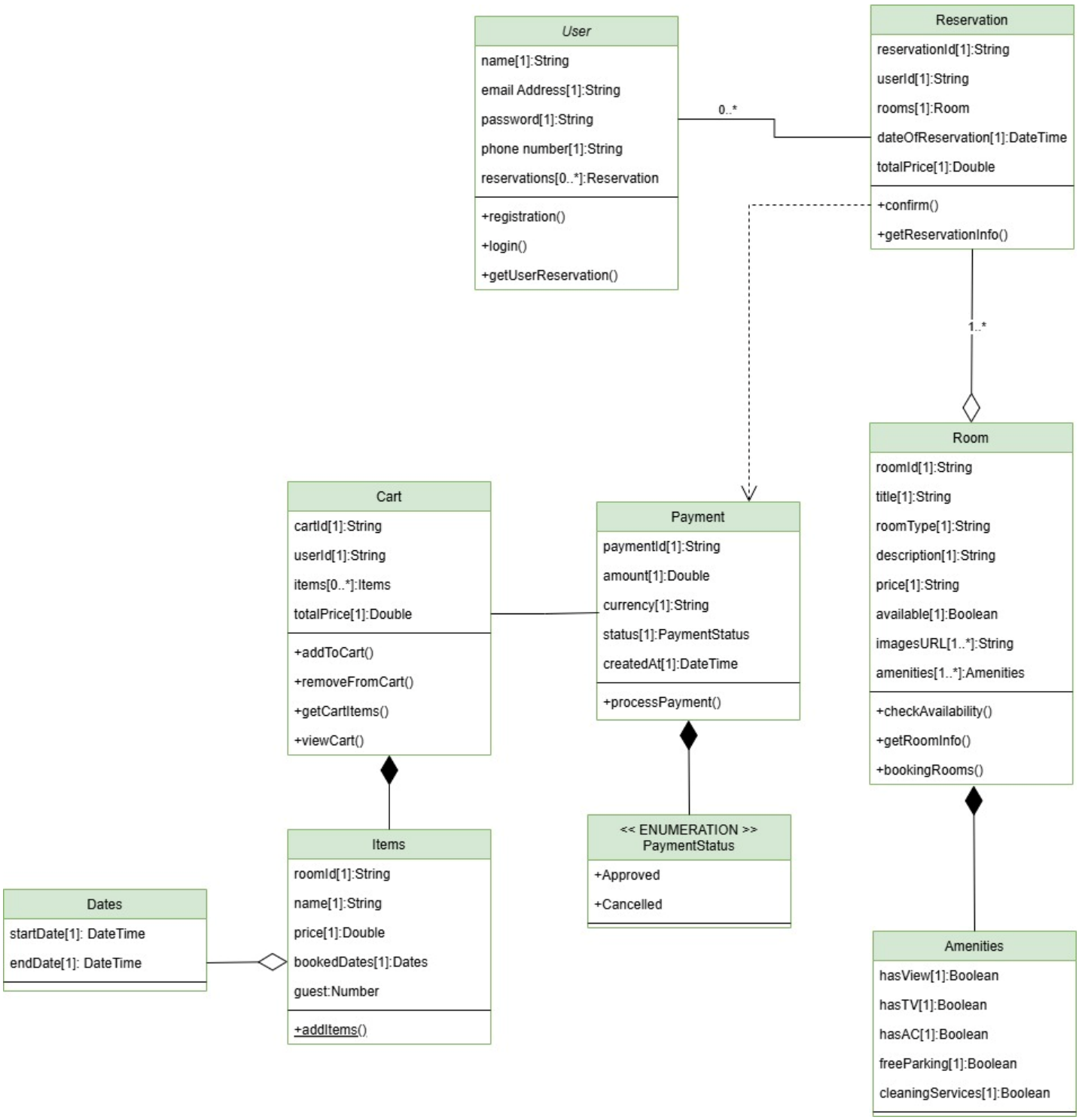
Субект	Действие	Обект
Нерегистриран потребител	преглежда	помещения
Нерегистриран потребител	преглежда	галерия на хотела
Нерегистриран потребител	преглежда	данни за хотела и локация
Нерегистриран потребител	взаимодейства	с картата с локация
Нерегистриран потребител	търси	помещения
Нерегистриран потребител	филтрира	помещения
Нерегистриран потребител	регистрира	профил като клиент
Нерегистриран потребител	въвежда	необходимите данни
Google maps	визуализира	карта с локация
Регистриран потребител	влиза	в профила си в системата
Регистриран потребител	преглежда	история с резервации
Регистриран потребител	преглежда	галерия на хотела
Регистриран потребител	преглежда	данни за хотела и локация
Регистриран потребител	взаимодейства	с картата с локация
Регистриран потребител	преглежда	количката
Регистриран потребител	преглежда	помещения
Регистриран потребител	търси	помещения
Регистриран потребител	филтрира	помещения
Регистриран потребител	въвежда	дати във формата за регистрация
Регистриран потребител	добавя	стаи в количката
Регистриран потребител	плаща	общата сума в количката
Регистриран потребител	попълва	данни за плащане
Stripe	оторизира	плащането

Гр. София
Януари 2025

Начален списък на потребителските случаи			
Име	*Главен актьор*	*Кратко описание*	*Приоритет*
Добавяне на стая в количката	Регистриран потребител	Потребителят е в страницата на конкретна стая. Визуализирана е форма за попълване на дата на настаняване, дата на напускане и брой гости. Най-отгоре на формата е показана цената за 1 нощувка и при избор на период най-отдолу във формата се пресмята общата цена за престоя. Ако потребителят е съгласен с цената, натиска върху бутона "Add to cart".	H
Осъществяване на плащане	Регистриран потребител	Потребителят натиска върху бутона "Chekout", който се намира в страницата на количката. При клик върху бутона се отваря форма за плащане, в която потребителят трябва да въведе данни за своята карта. Когато всички данни са валидни натиска върху бутона "Confirm and Pay" и ако всичко е изрядно плащането е успешно и потребителят е пренасочен към своя профил за да види направената резервация.	H
Преглеждане на стаи	Регистриран потребител, Нерегистриран потребител	Потребителят избира конкретната стая. Натиска върху нея и се отваря нова страница, на която се визуализира името на стаята, описание, удобства, цена, снимки и форма за попълване на резервационни данни.	H
Търсене на стаи	Регистриран потребител, Нерегистриран потребител	Потребителят избира опцията "Room", която се намира в навигационното меню. Визуализира се страница с всички налични стаи, от които клиентът може да избере коя да резервира. За стаите са показани имената и цените им за една нощувка.	H
Филтриране на стаи	Регистриран потребител, Нерегистриран потребител	Потребителят отива на началната страница на хотела и вижда сърч бар (поле за въвеждане на дати и брой гости). Въвежда желания от него период за отсядане в хотела, както и броят гости. Натиска върху бутона "Search". Отваря се нова страница, на която са заредени само стаите, които отговарят на зададените филтри.	H
Интеракция с картата с локация	Регистриран потребител, Нерегистриран потребител	Потребителят се намира в страницата "About Us". На нея е визуализирана карта, с която потребителят може да взаимодейства. Той може да променя нейният изглед - да бъде map или satellite, да променя големината на картата и мащаба. В дясно на картата има и човече, което потребителят може да пусне на произволно място на картата, за да види какво точно има в лизост до хотела.	H
Преглед на количка	Регистриран потребител	Потребителят избира "Cart" в навигационното меню. Отваря се нова страница с всички добавени стаи, общата цена за плащане и бутон за преминаване към плащане.	H
Въвеждане на дати във формата за резервация	Регистриран потребител	Потребителят е на страницата с информация за конкретна стая. Там е визуализирана форма, в която потребителят въвежда желаният период за престой, както и броят гости.	H
Преглеждане на история с резервации	Регистриран потребител	Потребителят натиска върху своя имейл, намиращ се в дясната част на навигационното меню. Отваря се нова страница с профила на потребителят, в който се визуализират всички направени резервации под формата на картички. Те съдържат информация за стаята, която е запазена, цена, дати и номер на направената резервация.	H
Преглеждане на галерия	Регистриран потребител, Нерегистриран потребител	Потребителят избира опцията "Gallery", която се намира в навигационното меню. Визуализира се страница с всички снимки на хотела, които потребителят може да разгледа.	M
Преглеждане на данни за хотела и локация	Регистриран потребител, Нерегистриран потребител	Потребителят избира опцията "About Us", която се намира в навигационното меню. Визуализира се страница с данни за контакт с хотела и интерактивна карта с локация.	M
Попълване на данни за плащане	Регистриран потребител	Попълва формата за плащане с валидни данни за своята карта.	L
Вход в системата	Регистриран потребител	Въвежда имейл и парола за влизане в системата	L
Регистрация в системата	Нерегистриран потребител	Въвежда необходимите данни за да осъществи регистрация - име, имейл, парола и телефонен номер	L
Въвеждане на данни за регистрация	Нерегистриран потребител	Въвежда необходимите данни в посочените полета като спазва изискванията	L

Гр. София
Януари 2025

6. UML диаграми на класовете



Гр. София
Януари 2025

7. Описание на базата данни

1) Описание на User

Поле	Тип	Описание
`name`	`String`	Името на потребителя. Това е задължително поле.
`email`	`String`	Имейл адресът на потребителя. Изисква се и е уникален (не е посочено тук, но се препоръчва).
`password`	`String`	Паролата на потребителя, съхранена като хеширана стойност. Трябва да бъде минимум 6 символа.
`phoneNumber`	`String`	Телефонният номер на потребителя. Това поле също е задължително.
`reservations`	`Array<Object>`	Масив от обекти, всеки от които представлява информация за резервация, направена от потребителя.

В reservations се включва информация за всяка резервация, направена от потребителя.

Поле	Тип	Описание
`name`	`String`	Името на стаята, която е резервирана. Това поле е задължително.
`price`	`Number`	Цената на стаята за престоя. Това поле също е задължително.
`dates`	`Object`	Обект, съдържащ началната и крайната дата на престоя.
`dates.startDate`	`String`	Начална дата на резервацията. Задължително поле.
`dates.endDate`	`String`	Крайна дата на резервацията. Задължително поле.
`guests`	`Number`	Брой гости за тази резервация. Това поле е задължително.

2) Описание на Room

Поле	Тип	Описание
`title`	`String`	Заглавие на стаята (напр. "Луксозна стая"). Това е задължително поле.
`roomType`	`String`	Тип на стаята (единична, двойна, апартамент и т.н.). Задължително поле.
`description`	`String`	Подробно описание на стаята. Това поле също е задължително.
`price`	`Number`	Цена на нощувка. Трябва да бъде число по-голямо или равно на 0.
`available`	`Boolean`	Показва дали стаята е налична за резервация. Това е задължително поле.
`image`	`Object`	Обект, съдържащ URL линкове към изображения на стаята. Всички подполета са задължителни.
`amenities`	`Object`	Обект, описващ удобствата в стаята. Всички подполета са задължителни.
`bookedPeriods`	`Array<Object>`	Масив от обекти, описващи резервираните периоди за стаята.

Гр. София
Януари 2025

Подполетата в image показват url на всяка добавена снимка.

Поле	Тип	Описание
`first`	`String`	Линк към първото изображение на стаята.
`second`	`String`	Линк към второто изображение на стаята.
`third`	`String`	Линк към третото изображение на стаята.

Подполета в amenities, показват кои удобства са налични законкретната стая.

Поле	Тип	Описание
`hasView`	`Boolean`	Дали стаята разполага с изглед (напр. към море, планина).
`hasTV`	`Boolean`	Дали стаята разполага с телевизор.
`hasAC`	`Boolean`	Дали стаята разполага с климатик.
`freeParking`	`Boolean`	Дали стаята включва безплатен паркинг.
`cleaningServices`	`Boolean`	Дали стаята предлага почистващи услуги.

Подполетата в bookedPeriods Показват периодите, за които е резервирана стаята.

Поле	Тип	Описание
`startDate`	`String`	Начална дата на резервацията. Това е задължително поле.
`endDate`	`String`	Крайна дата на резервацията. Това е задължително поле.

3)Описание на Cart

Поле	Тип	Описание
`userId`	`String`	Уникален идентификатор на потребителя, който притежава кошницата. Задължително поле.
`items`	`Array<Object>`	Масив от обекти, представляващи избраните стаи и свързаната информация.

Полето items съдържат информация за всяка стая, добавена в кошницата.

Поле	Тип	Описание
`roomId`	`ObjectId`	Референция към стаята в колекцията `Room`. Задължително поле.
`name`	`String`	Името на стаята. Задължително поле.
`price`	`Number`	Цена на стаята за една нощувка. Задължително поле.
`dates`	`Object`	Обект, съдържащ началната и крайната дата на престоя.
`dates.startDate`	`String`	Начална дата на резервацията. Задължително поле.
`dates.endDate`	`String`	Крайна дата на резервацията. Задължително поле.
`guests`	`Number`	Брой гости за избраната стая. Задължително поле.

4)Описание на Reservation

Поле	Тип	Описание
`userId`	`ObjectId`	Референция към потребител от колекцията `User`. Задължително поле.
`rooms`	`Array<Object>`	Масив от обекти, представляващи стаите, включени в резервацията.
`dateOfReservation`	`Date`	Дата на създаване на резервацията. По подразбиране се задава текущата дата.
`totalPrice`	`Number`	Обща сума за резервацията. Това поле е задължително.

Полетата в масива rooms съдържат информация за всяка стая, включена в резервацията.

Поле	Тип	Описание
`roomId`	`ObjectId`	Референция към стаята в колекцията `Room`. Задължително поле.
`name`	`String`	Името на стаята. Задължително поле.
`price`	`Number`	Цена на стаята за една нощувка. Задължително поле.
`startDate`	`String`	Начална дата на престоя. Задължително поле.
`endDate`	`String`	Крайна дата на престоя. Задължително поле.

5)Описание на Payment

Поле	Тип	Описание
`paymentId`	`String`	Уникален идентификатор на плащането. Задължително поле.
`amount`	`Number`	Сумата, свързана с плащането. Задължително поле.
`currency`	`String`	Валута на плащането (напр. "USD", "EUR"). Задължително поле.
`status`	`String`	Статус на плащането (напр. "completed", "pending", "failed"). Задължително поле.
`createdAt`	`Date`	Дата на създаване на плащането. По подразбиране се задава текущата дата.

За всяка схема има генерирано уникално _id от MongoDB, което не е показано в приложените таблици.

Общ преглед на връзките:

1. **User ↔ Reservation:**
   - Един потребител може да има множество резервации.
2. **Reservation ↔ Room:**
   - Една резервация може да включва множество стаи.
3. **Reservation ↔ Payment:**
   - Всяка резервация има едно плащане.
4. **Cart ↔ Room:**
   - Кошницата може да съдържа множество стаи.

8. <u>Описание на потребителската кошница</u>

1)В бекенда

В бекенда, потребителската кошница е представена чрез MongoDB схема, наречена Cart. Тази схема съдържа информация за потребителя, който притежава кошницата, както и данни за избраните стаи. Схемата на кошницата е предоставена в предишната точка:

```
const mongoose = require('mongoose');

const Schema = mongoose.Schema;

const CartSchema = new mongoose.Schema({
    userId: { type: String, required: true },
    items: [
        {
```

```
            roomId: { type: mongoose.Schema.Types.ObjectId, required: true,
ref: 'Room' },
            name: { type: String, required: true },
            price: { type: Number, required: true },
            dates: {
                startDate: { type: String, required: true },
                endDate: { type: String, required: true },
            },
            guests: { type: Number, required: true }
        },
    ]
});

module.exports = mongoose.model('Cart', CartSchema);
```

Чрез тази структура осигуряваме централизирано съхранение на данните за кошницата, което улеснява извличането и управлението ѝ.

API методи за работа с кошницата:

1. Добавяне на стая в кошницата

Методът за добавяне позволява на потребителя да избере стая и да я добави към кошницата си. При извикването на API маршрута POST /cart/add, сървърът:

1. Проверява дали вече съществува кошница за дадения потребител.
2. Ако няма, създава нова кошница.
3. Добавя информацията за избраната стая към масива items.

2. Извличане на кошницата

Методът за извличане на кошницата (GET /cart/:userId) връща всички текущи елементи в кошницата на потребителя. Това позволява на фронтенда да показва избраните стаи в реално време.

3. Премахване на стая от кошницата

Методът за премахване на стая (DELETE /cart/remove) позволява на потребителя да изтрива конкретна стая от кошницата. Сървърът намира кошницата на потребителя и премахва съответния обект от масива items.

2)Във фронтенда

Фронтендът осигурява интуитивен интерфейс за управление на кошницата, като използва React компоненти. Потребителят може да преглежда избраните стаи, добавя нови стаи в кошницата, премахва стаи от кошницата.

Този компонент (Cart.jsx) визуализира съдържанието на кошницата. При зареждане, той извиква API метода за извличане на кошницата и показва списък с избраните стаи, като предоставя бутон за премахване на всяка стая.

В SingleRoom.jsx се намира бутонът за добавяне на стая в кошницата като отново се извиква API метод.

9. <u>Описание на плащането</u>

-Конвейер за обработка на поръчките включва няколко автоматизирани стъпки:

1. Валидация на данните:
   - Проверка на коректността на данните, въведени от потребителя (например валидност на дати и броя на гостите).
2. Обработка на плащането:
   - След като данните бъдат валидирани, поръчката преминава към обработка на плащането.
3. Потвърждение на резервацията:
   - След успешно плащане, системата генерира потвърждение за резервацията и го свързва с потребителския профил.

-Описание на плащането - Плащането Се реализира чрез колекцията Payment, която съхранява информация за всяко извършено плащане.

1. Инициализиране на плащането:
   - Създава се нов запис в колекцията Payment с уникален идентификатор (paymentId), сумата на плащането (amount), валутата (currency) и текущата дата (createdAt).
2. Обработка на плащането:
   - Ако проектът използва платежен процесор (например Stripe), бекендът изпраща данните за плащането към API на Stripe.
   - Платежният процесор проверява валидността на картата, сумата и сигурността.
3. Оторизация на кредитната карта:
   - Ако оторизацията е успешна, платежният процесор връща статус completed, който се записва в полето status на плащането.
   - Ако плащането се провали, статусът се задава като failed, а резервацията не се създава.
4. Свързване на плащането с резервацията:
   - След успешна обработка на плащането, то се свързва с резервацията чрез идентификатор на резервацията (reservationId).

Управлението се случва в следната функция:

```
const handleCheckout = async (req, res, next) => {

    const decodedToken = verifyToken(req);
    const userId = decodedToken.userId;

    try {
        const { userId, rooms, amount, currency } = req.body;

        const cart = await Cart.findOne({ userId }).populate('items.roomId');
        if (!cart || cart.items.length === 0) {
            return res.status(400).json({ message: 'Your cart is empty.' });
        }

        if (!amount || !currency) {
            return res.status(400).json({ message: 'Invalid input data' });
        }

        const paymentIntent = await stripe.paymentIntents.create({
            amount,
            currency: 'eur',
        });

        console.log('Payment successful:', paymentIntent);

        const payment = new Payment({
            paymentId: paymentIntent.id,
            amount,
            currency,
            status: paymentIntent.status,
        });
        await payment.save();

        const processedRooms = [];
        const errors = [];

        console.log('Processing rooms:', rooms);
        rooms.forEach((item, index) => {
            if (!item.roomId || !item.name || !item.startDate || !item.endDate
|| !item.price) {
                errors.push({
                    index,
                    message: 'Cart item is missing required fields.',
                    missingFields: {
                        roomId: !item.roomId,
                        name: !item.name,
                        price: !item.price,
```

```
                    startDate: !item.startDate,
                    endDate: !item.endDate,
                },
            });
        } else {
            processedRooms.push({
                roomId: item.roomId,
                name: item.name,
                price: item.price,
                startDate: item.startDate,
                endDate: item.endDate,
            });
        }
    });

    if (errors.length > 0) {
        console.error('Invalid cart items detected:', errors);
        return res.status(400).json({
            message: 'Some cart items are invalid.',
            errors,
        });
    }

    console.log('Processed rooms:', processedRooms);

    const totalPrice = rooms.reduce((sum, room) => sum + room.price, 0);

    const newReservation = new Reservation({
        userId,
        rooms: processedRooms,
        dateOfReservation: new Date(),
        totalPrice,
    });

    await newReservation.save();

    cart.items = [];
    await cart.save();

    res.status(200).json({
        message: 'Checkout and reservation completed successfully.',
        reservation: newReservation,
        clientSecret: paymentIntent.client_secret,
    });
} catch (err) {
    console.error('Error during checkout and reservation:', err);
    next(new HttpError('Checkout failed.', 500));
```

```
    }
};
```

10. Стартиране на проекта

1)Разархивирайте архива

2)Отворете папката Project в Visual Studio Code

2)Отворете папката myproject чрез cd myproject напишете npm install в терминала

3)След това в терминала напишете npm start

4) Отворете папката Backend в друг Visual Studio Code прозорец

5) Напишете npm install в терминала

6)След това в теминала напишете Npm start

- Фронтенда работи http://localhost:3000, а бекенда на http://localhost:5000